Top Module

```
module top_module(
input clk, rst,
input [31:0] A,
input [31:0] B,
input [1:0] sel,
output [31:0] accumulator
);

reg [31:0] acc32_reg;
reg [15:0] acc16_reg;
reg [15:0] accbf_reg;

wire [31:0] C_32mul;
wire [15:0] C_16mul;
wire [15:0] C_16mulbf;

wire [31:0] C_32add;
wire [15:0] C_16add;
wire [15:0] C_bfadd;

reg [2:0] enable;

always @(*) begin
 case(sel)
  2'b00: enable = 3'b001;
  2'b01: enable = 3'b010;
  2'b10: enable = 3'b100;
  default: enable = 3'b000;
 endcase
end

always @(posedge clk) begin
 if(rst) begin
   acc32_reg <= 0;
   acc16_reg <= 0;
   accbf_reg <= 0;
 end else begin
   if(enable[0]) acc32_reg <= C_32add;
   if(enable[1]) acc16_reg <= C_16add;
   if(enable[2]) accbf_reg <= C_bfadd;
 end
end

fp32_mul M0 (.en(enable[0]), .a(A), .b(B), .c(C_32mul));
fp32_add A0 (.en(enable[0]), .a(C_32mul), .b(acc32_reg), .c(C_32add));

fp16_mul M1 (.en(enable[1]), .a(A[15:0]), .b(B[15:0]), .c(C_16mul));
fp16_add A1 (.en(enable[1]), .a(C_16mul), .b(acc16_reg), .c(C_16add));

bf16_mul M2 (.en(enable[2]), .a(A[15:0]), .b(B[15:0]), .c(C_16mulbf));
bf16_add A2 (.en(enable[2]), .a(C_16mulbf), .b(accbf_reg), .c(C_bfadd));

assign accumulator =
 (sel==2'b00)?acc32_reg:
 (sel==2'b01)?{16'b0,acc16_reg}:
 (sel==2'b10)?{16'b0,accbf_reg}:acc32_reg;

endmodule
```

FP32 Multiplier

```
module fp32_mul(input en,input [31:0] a,input [31:0] b,output [31:0] c);
reg [31:0] c_reg;
wire [23:0] mant_a=(a[30:23]==0)?{1'b0,a[22:0]}:{1'b1,a[22:0]};
wire [23:0] mant_b=(b[30:23]==0)?{1'b0,b[22:0]}:{1'b1,b[22:0]};
reg [47:0] mant_prod; reg [8:0] exp_temp;

always @(*) begin
 if(en) begin
  c_reg[31]=a[31]^b[31];
  exp_temp=a[30:23]+b[30:23]-127;
  mant_prod=mant_a*mant_b;
  if(mant_prod[47])
     c_reg[30:0]={exp_temp+1,mant_prod[46:24]};
  else
     c_reg[30:0]={exp_temp,mant_prod[45:23]};
 end else c_reg=0;
end
assign c=c_reg;
endmodule
```

FP32 Adder

```
module fp32_add(input en,input [31:0] a,input [31:0] b,output [31:0] c);
reg [31:0] c_reg;
reg [23:0] mant_a,mant_b;
reg [24:0] mant_big,mant_small,mant_sum;
reg [7:0] exp_a,exp_b,exp_out;
reg sign_a,sign_b,sign_out;
reg [7:0] diff;
integer i; reg found;

always @(*) begin
 if(!en) c_reg=c_reg;
 else begin
  sign_a=a[31]; sign_b=b[31];
  exp_a=a[30:23]; exp_b=b[30:23];
  mant_a=(exp_a==0)?{1'b0,a[22:0]}:{1'b1,a[22:0]};
  mant_b=(exp_b==0)?{1'b0,b[22:0]}:{1'b1,b[22:0]};

  if(exp_a>exp_b) begin
    diff=exp_a-exp_b; exp_out=exp_a;
    mant_big={1'b0,mant_a};
    mant_small={1'b0,mant_b}>>diff;
    sign_out=sign_a;
  end else begin
    diff=exp_b-exp_a; exp_out=exp_b;
    mant_big={1'b0,mant_b};
    mant_small={1'b0,mant_a}>>diff;
    sign_out=sign_b;
  end

  mant_sum=(sign_a==sign_b)?mant_big+mant_small:mant_big-mant_small;

  if(mant_sum[24]) begin
    mant_sum=mant_sum>>1;
    exp_out=exp_out+1;
  end else begin
    found=0;
    for(i=23;i>=0;i=i-1)
      if(!found && mant_sum[i]) begin
        mant_sum=mant_sum<<(23-i);
        exp_out=exp_out-(23-i);
        found=1;
      end
  end

  c_reg={sign_out,exp_out,mant_sum[22:0]};
 end
end
assign c=c_reg;
endmodule
```

FP16 Multiplier

```
module fp16_mul(input en,input [15:0] a,input [15:0] b,output [15:0] c);
reg [15:0] c_reg;
wire [10:0] mant_a=(a[14:10]==0)?{1'b0,a[9:0]}:{1'b1,a[9:0]};
wire [10:0] mant_b=(b[14:10]==0)?{1'b0,b[9:0]}:{1'b1,b[9:0]};
reg [21:0] mant_prod; reg [5:0] exp_temp;

always @(*) begin
 if(en) begin
  c_reg[15]=a[15]^b[15];
  exp_temp=a[14:10]+b[14:10]-15;
  mant_prod=mant_a*mant_b;
  if(mant_prod[21])
     c_reg[14:0]={exp_temp+1,mant_prod[20:11]};
  else
     c_reg[14:0]={exp_temp,mant_prod[19:10]};
 end else c_reg=0;
end
assign c=c_reg;
endmodule
```

FP16 Adder

```
module fp16_add(input en,input [15:0] a,input [15:0] b,output [15:0] c);
reg [15:0] c_reg;
reg [10:0] mant_a,mant_b;
reg [11:0] mant_big,mant_small,mant_sum;
reg [4:0] exp_a,exp_b,exp_out;
reg sign_a,sign_b,sign_out;
reg [4:0] diff; integer i; reg found;

always @(*) begin
 if(!en) c_reg=c_reg;
 else begin
  sign_a=a[15]; sign_b=b[15];
  exp_a=a[14:10]; exp_b=b[14:10];
  mant_a=(exp_a==0)?{1'b0,a[9:0]}:{1'b1,a[9:0]};
  mant_b=(exp_b==0)?{1'b0,b[9:0]}:{1'b1,b[9:0]};

  if(exp_a>exp_b) begin
    diff=exp_a-exp_b; exp_out=exp_a;
    mant_big={1'b0,mant_a};
    mant_small={1'b0,mant_b}>>diff;
    sign_out=sign_a;
  end else begin
    diff=exp_b-exp_a; exp_out=exp_b;
    mant_big={1'b0,mant_b};
    mant_small={1'b0,mant_a}>>diff;
    sign_out=sign_b;
  end

  mant_sum=(sign_a==sign_b)?mant_big+mant_small:mant_big-mant_small;

  if(mant_sum[11]) begin
    mant_sum=mant_sum>>1;
    exp_out=exp_out+1;
  end else begin
    found=0;
    for(i=10;i>=0;i=i-1)
      if(!found && mant_sum[i]) begin
        mant_sum=mant_sum<<(10-i);
        exp_out=exp_out-(10-i);
        found=1;
      end
  end

  c_reg={sign_out,exp_out,mant_sum[9:0]};
 end
end
assign c=c_reg;
endmodule
```

BF16 Multiplier

```
module bf16_mul(input en,input [15:0] a,input [15:0] b,output [15:0] c);
reg [15:0] c_reg;
wire [7:0] mant_a=(a[14:7]==0)?{1'b0,a[6:0]}:{1'b1,a[6:0]};
wire [7:0] mant_b=(b[14:7]==0)?{1'b0,b[6:0]}:{1'b1,b[6:0]};
reg [15:0] mant_prod; reg [8:0] exp_temp;

always @(*) begin
 if(en) begin
  c_reg[15]=a[15]^b[15];
  exp_temp=a[14:7]+b[14:7]-127;
  mant_prod=mant_a*mant_b;
  if(mant_prod[15])
     c_reg[14:0]={exp_temp+1,mant_prod[14:8]};
  else
     c_reg[14:0]={exp_temp,mant_prod[13:7]};
 end else c_reg=0;
end
assign c=c_reg;
endmodule
```

BF16 Adder

```
module bf16_add(input en,input [15:0] a,input [15:0] b,output [15:0] c);
reg [15:0] c_reg;
reg sign_a,sign_b,sign_out;
reg [7:0] exp_a,exp_b,exp_out;
reg [7:0] exp_diff;
reg [7:0] mant_a,mant_b;
reg [8:0] mant_big,mant_small,mant_sum;
integer i; reg found_one;

always @(*) begin
 sign_a=a[15]; sign_b=b[15];
 exp_a=a[14:7]; exp_b=b[14:7];
 mant_a=(exp_a==0)?{1'b0,a[6:0]}:{1'b1,a[6:0]};
 mant_b=(exp_b==0)?{1'b0,b[6:0]}:{1'b1,b[6:0]};

 if(exp_a>exp_b) begin
   exp_diff=exp_a-exp_b;
   exp_out=exp_a;
   mant_big={1'b0,mant_a};
   mant_small={1'b0,mant_b}>>exp_diff;
   sign_out=sign_a;
 end else begin
   exp_diff=exp_b-exp_a;
   exp_out=exp_b;
   mant_big={1'b0,mant_b};
   mant_small={1'b0,mant_a}>>exp_diff;
   sign_out=sign_b;
 end

 mant_sum=(sign_a==sign_b)?mant_big+mant_small:mant_big-mant_small;

 if(mant_sum[8]) begin
   mant_sum=mant_sum>>1;
   exp_out=exp_out+1;
 end else begin
   found_one=0;
   for(i=7;i>=0;i=i-1)
     if(!found_one && mant_sum[i]) begin
       mant_sum=mant_sum<<(7-i);
       exp_out=exp_out-(7-i);
       found_one=1;
     end
 end

 c_reg={sign_out,exp_out,mant_sum[6:0]};
end
assign c=c_reg;
endmodule
```